

Java Streams

What is Stream?

A **Stream** in Java is a sequence of elements that support functional-style operations.

- It does NOT store data.
- It does NOT modify the original collection.
- It processes data via a **pipeline**.

Stream Pipeline Structure

Source → Intermediate Operations → Terminal Operation

Types of Operations

Intermediate Operations (Lazy)

Return Stream:

- filter()
- map()
- distinct()
- sorted()
- skip()
- limit()

Terminal Operations (Trigger execution)

Return final result:

- collect()
- reduce()
- count()
- min()
- max()
- findFirst()
- findAny()
- anyMatch()
- allMatch()
- noneMatch()
- average()
- sum()

STREAMS ON PRIMITIVE & STRING COLLECTIONS

```
List<Integer> arr = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
```

FILTER (Intermediate Operation)

Even Numbers & Odd Numbers → Filter

```
List<Integer> even = arr.stream()
    .filter(n -> n % 2 == 0)
    .toList();

List<Integer> odd = arr.stream()
    .filter(n -> n % 2 != 0)
    .collect(Collectors.toList());
```

Square of Even Numbers > 5 → Filter + Map

```
List<Integer> sqevn5 = arr.stream()
    .filter(n -> n > 5)
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .toList();
```

REDUCE (Terminal Operation)

Summing All Numbers

```
int sum = arr.stream()
    .reduce(0, (a, b) -> a + b);

int sumNoIdentity = arr.stream()
    .reduce(0, Integer::sum);

Optional<Integer> sumopt = arr.stream()
    .reduce((a, b) -> a + b);
```

Using mapToInt()

```
int summ = arr.stream()
    .mapToInt(a -> a)
    .sum();
```

MAX / MIN

```
int maxx = arr.stream()
    .max(Integer::compareTo)
    .get();

int minn = arr.stream()
    .min(Integer::compareTo)
    .get();
```

Safer version:

```
List<Integer> arr2 = Arrays.asList();

int min2 = arr2.stream()
    .min(Integer::compareTo)
    .orElse(-1);
```

Using reduce for max/min

```
Integer maxx = arr.stream()
    .reduce(Integer.MIN_VALUE, Integer::max);

Optional<Integer> maxxOpt = arr.stream()
    .reduce(Integer::max);

Integer maxxOrElse = arr.stream()
    .reduce(Integer::max)
    .orElse(Integer.MIN_VALUE);
```

distinct()

Removes duplicate elements.

```
List<Integer> arr3 = Arrays.asList(1,1,2,3,5,6,7);

List<Integer> dis = arr3.stream()
    .distinct()
    .toList();
```

sorted()

Ascending

```
List<Integer> arr4 = Arrays.asList(1,4,3,2,5,7,8,6,9);

List<Integer> sortAsc = arr4.stream()
    .sorted()
    .toList();
```

Descending

```
List<Integer> sortDesc = arr4.stream()
    .sorted(Comparator.reverseOrder())
    .toList();
```

Combined Operations

Sum of even numbers

```
int sumEven = arr.stream()
    .filter(n -> n % 2 == 0)
    .reduce(0, Integer::sum);
```

Sum of odd numbers

```
int sumOdd = arr.stream()
    .filter(n -> n % 2 != 0)
    .reduce(0, Integer::sum);
```

Sum of Squares of Even Numbers

```
int sumEvnSq = arr.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .reduce(0, Integer::sum);
```

Safer Optional version:

```
List<Integer> arr5 = Arrays.asList();

Optional<Integer> result = arr5.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .reduce(Integer::sum);

int res = arr5.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .reduce(Integer::sum)
    .orElse(-1);
```

Product Using reduce()

```
int prod = arr.stream()
    .reduce(1, (a, b) -> a * b);

prod = arr.stream()
    .reduce((a, b) -> a * b)
    .orElse(-1);
```

findFirst()

First Element greater than 7, greater than 1000

```
int eleGt7 = arr.stream()
    .filter(n -> n > 7)
    .findFirst()
    .get();

int eleGt1000 = arr.stream()
    .filter(n -> n > 1000)
    .findFirst()
    .orElse(-1);

Optional<Integer> eleGt1000Opt =
    arr.stream()
        .filter(n -> n > 1000)
        .findFirst();
```

anyMatch / allMatch / noneMatch

Any element (>10), All are even, none of them are negative

```
List<Integer> nums = Arrays.asList(3,5,7,12,4);

boolean result_check = nums.stream()
    .anyMatch(n -> n > 10);

boolean even_check = nums.stream()
    .allMatch(n -> n % 2 == 0);

boolean negative_check = nums.stream()
    .noneMatch(n -> n < 0);
```

count()

```
long count = nums.stream()
    .filter(n -> n > 5)
    .count();
```

Convert List to Map → String and it's Length

```
List<String> names = Arrays.asList("Surya","Mahesh","Kiran");

Map<String,Integer> map = names.stream()
    .collect(Collectors.toMap(
        name -> name,
        name -> name.length()
    ));
```

Advanced Reduce Practice

Product

```
int product = nums.stream()
    .reduce(1, (a, b) -> a * b);
```

Sum of Squares

```
int sumOfSquares = nums.stream()
    .map(n -> n * n)
    .reduce(0, Integer::sum);
```

Longest String

```
Optional<String> longest_string =  
    names.stream()  
        .reduce((a, b) ->  
            a.length() > b.length() ? a : b);
```

Advanced Sorting & Finding

```
List<Integer> numbers = Arrays.asList(  
    1,1,1,1,5,7,73,2,240,111,2,3,7,6,-1,100,45,120,240,111);
```

Least

```
int least = numbers.stream()  
    .sorted()  
    .findFirst()  
    .orElse(-1);
```

Second Highest

```
int second_highest = numbers.stream()  
    .sorted(Comparator.reverseOrder())  
    .skip(1)  
    .findFirst()  
    .orElse(-1);
```

Distinct + Count + Sort

Distinct Numbers list & Count of Distinct Numbers

```
List<Integer> distinct_numbers =  
    numbers.stream()  
        .distinct()  
        .toList();  
  
long distinct_num_count =  
    numbers.stream()  
        .distinct()  
        .count();
```

Sort all distinct elements in list

```
List<Integer> distinct_sort =  
    numbers.stream()  
        .distinct()  
        .sorted()  
        .toList();
```

String Filtering

```
String namewiths = names.stream()  
    .filter(s -> s.startsWith("s"))  
    .findFirst()  
    .orElse("");  
  
String nameendswithn = names.stream()  
    .filter(s -> s.endsWith("n"))  
    .findFirst()  
    .orElse("");
```

PART 2 – ClassStreams (Employee Example)

```
class Employee {  
    private int id;  
    private String name;  
    private int age;  
    private String department;  
    private double salary;  
  
    public Employee(int id, String name,  
                    int age, String department,  
                    double salary) {  
        this.id = id;  
        this.name = name;  
        this.age = age;  
        this.department = department;  
        this.salary = salary;  
    }  
}
```



```

    public int getId() { return id; }
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }

    public String toString() {
        return id+" - "+name+" - "+age+" - "+department+" - "+salary;
    }
}

```

```

List<Employee> employees = Arrays.asList(
    new Employee(1,"Surya",25,"IT",60000),
    new Employee(2,"Mahesh",30,"HR",50000),
    new Employee(3,"Kiran",28,"IT",75000),
    new Employee(4,"Anil",35,"Finance",90000),
    new Employee(5,"Suresh",40,"IT",120000),
    new Employee(6,"Ravi",22,"HR",40000),
    new Employee(7,"John",29,"Finance",85000),
    new Employee(8,"Sam",31,"IT",95000)
);

```

Filtering

List of Employees whose Salary > 70000

```

employees.stream()
    .filter(e -> e.getSalary() > 70000)
    .toList();

```

List of all IT Employees

```

employees.stream()
    .filter(e -> e.getDepartment().equals("IT"))
    .toList();

```

⚠ Always use .equals() not == for Strings.

Map

List of all employee names

```
employees.stream()  
    .map(Employee::getName)  
    .toList();
```

List of all employee salaries

```
employees.stream()  
    .map(Employee::getSalary)  
    .toList();
```

Sum of Salaries

```
double sum_salary = employees.stream()  
    .map(Employee::getSalary)  
    .reduce(0.0, Double::sum);
```

Max / Min

Employee with Highest salary

```
Employee highest_emp =  
    employees.stream()  
        .max(Comparator.comparing(Employee::getSalary))  
        .orElse(null);
```

Employee with Least Age – Youngest Employee

```
Employee youngest_emp =  
    employees.stream()  
        .min(Comparator.comparing(Employee::getAge))  
        .orElse(null);
```

Sorting

Sort Employees – by Salary ascending order

```
employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary))
    .toList();
```

Sort Employees – by Salary descending order

```
employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .toList();
```

Sort Employees – sort by dept then sort by Salary ascending order

```
employees.stream()
    .sorted(Comparator
        .comparing(Employee::getDepartment)
        .thenComparing(Employee::getSalary))
    .toList();
```

findFirst / skip

Find employee with Second highest salary

```
Employee secondHighest =
    employees.stream()
        .sorted(Comparator.comparing(Employee::getSalary).reversed())
        .skip(1)
        .findFirst()
        .orElse(null);
```

Match Operations

```
boolean anyHighSalary =
    employees.stream()
        .anyMatch(e -> e.getSalary() > 70000);

boolean all_emp_age_gt20 =
    employees.stream()
        .allMatch(e -> e.getAge() > 20);
```

```
boolean none_sales_emp =  
    employees.stream()  
        .noneMatch(e -> e.getDepartment().equals("Sales"));
```

Average Salary of Employees

```
double avgSalary =  
    employees.stream()  
        .mapToDouble(Employee::getSalary)  
        .average()  
        .orElse(0);
```

Grouping – Group by Dept & Count Number of Employees in each department

```
Map<String,List<Employee>> groupByDept =  
    employees.stream()  
        .collect(Collectors.groupingBy(Employee::getDepartment));  
  
Map<String,Long> deptCount =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.counting()  
        ));
```